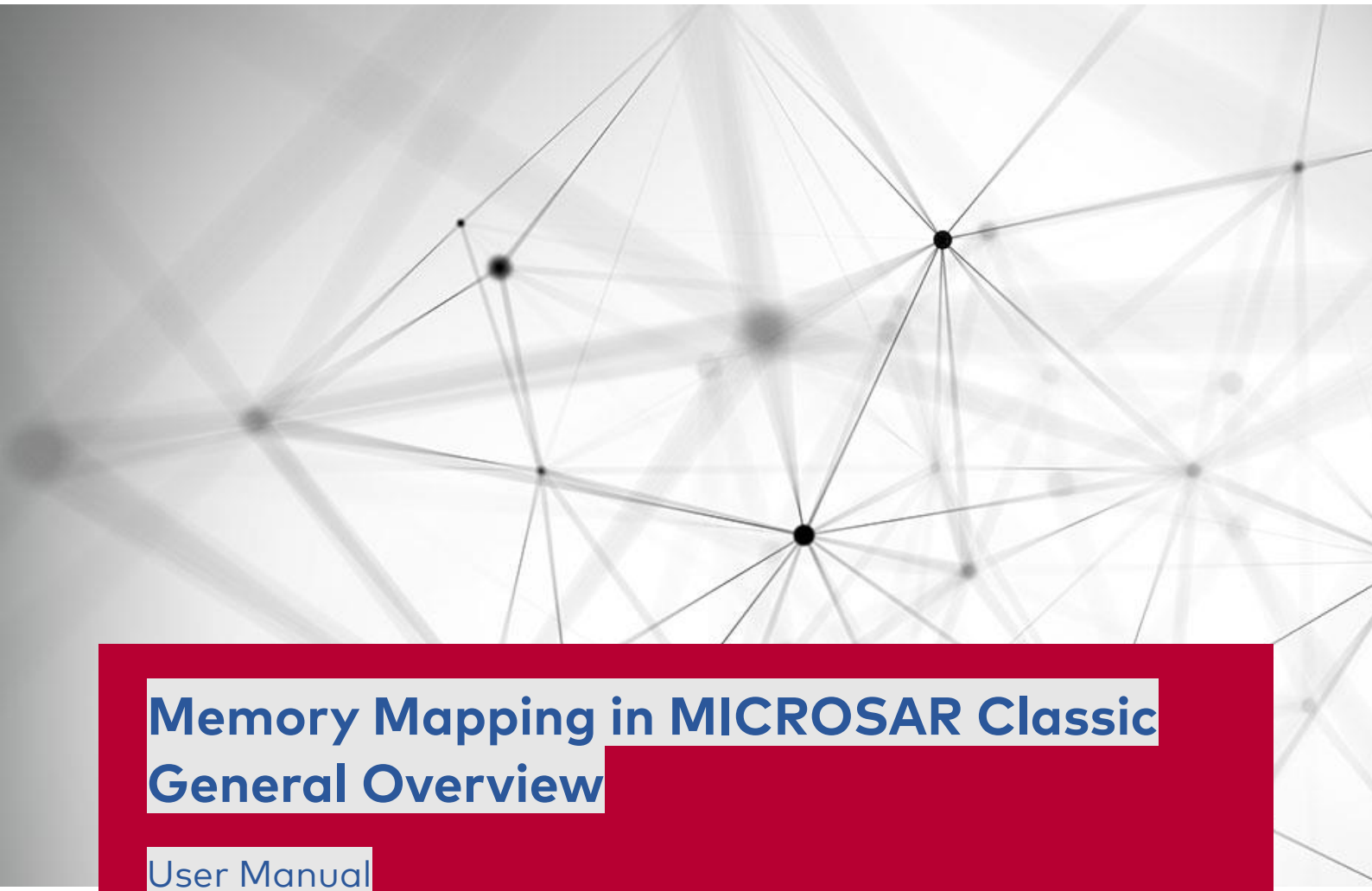
VECTOR



Memory Mapping in MICROSAR Classic General Overview

User Manual

Christian Kaiser

V1.0.0 | Released

Document Information

History

Author	Date	Version	Remarks
Christian Kaiser	2022-01-26	1.00.00	Initial Creation

Reference Documents

No.	Source	Title	Version
[1]	Vector	User Manual “MICROSAR Classic Post-Build Loadable”	2.0.4
[2]	Vector	Technical Reference “MICROSAR MemMap”	1.3.0
[3]	Vector	Technical Reference “MICROSAR vLinkGen”	2.5.0
[4]	Vector	User Manual “How to deal with MICROSAR Multi-Core Projects”	1.2.0
[5]	Vector	Application Note AN-ISC-8-1196 “Distributing BSW Components across Partitions”	1.2.0



Caution
This document gives an overview on product-wide behavior or use-cases. To ensure correct operation of the product always follow the instructions provided in the technical references and (additionally for safety projects) the safety manual. The content of these documents always replaces/overrides the content from this user manual.

Contents

1 Introduction 5

2 Overview 6
2.1 Common Aspects 6
2.2 “MemMap.h”-based components 7
2.3 Components with section descriptions 8
2.3.1 Conceptual Overview 9
2.3.1.1 SwAddrMethods 9
2.3.1.2 <Mip> Memory Sections 10
2.3.1.3 Custom Memory Sections 10
2.3.1.4 Addressing Mode Sets 11
2.3.1.5 MemMap Allocations 11
2.3.2 Module “MemMap” 11
2.4 vLinkGen 12

3 Configuration 14
3.1 Default configuration of MemMap 14
3.2 MemMap.h 14
3.3 MemMap_Common.h 14
3.4 Input to Configuration 15
3.5 Configuration Workflow 15
3.5.1 Target Hardware 15
3.5.2 Partitioning, ASIL Levels 16
3.5.2.1 Cross-Partition Data Exchange 16
3.6 Post-Build Configurations 17
3.7 Creating the Mapping 17

4 Glossary and Abbreviations 18
4.1 Glossary 18
4.2 Abbreviations 18

5 Contact 19

Illustrations

Figure 1 MICROSAR Classic MemMap concept – schematic view 6
Figure 2 – Conceptual overview of description and configuration elements for Memory Mapping..... 9
Figure 3 – Types/Sources of SwAddrMethods 10
Figure 4 MemMap Module (Generator) and vLinkGen (schematic) 12
Figure 5 Relationships between Memory Mapping and vLinkGen 13

1 Introduction

This document shall provide insight in MICROSAR Classic's Memory Mapping concept and its implementation.

Understanding our solution is crucial to configure a project according to its specific needs regarding memory mapping and allocation, especially in multi-core and multi-partition (ASIL) contexts.

2 Overview

Since release 27, MICROSAR Classic employs a hybrid memory mapping concept: It is designed to allow co-existence of modules using a central “MemMap.h” include file, along with modules describing their memory sections in AUTOSAR module description and relying on the existence of a (generated) “<Mip>_MemMap.h” file resulting from user’s configuration, as specified in AUTOSAR 4.3.

The elements with green border were newly introduced in release 27.

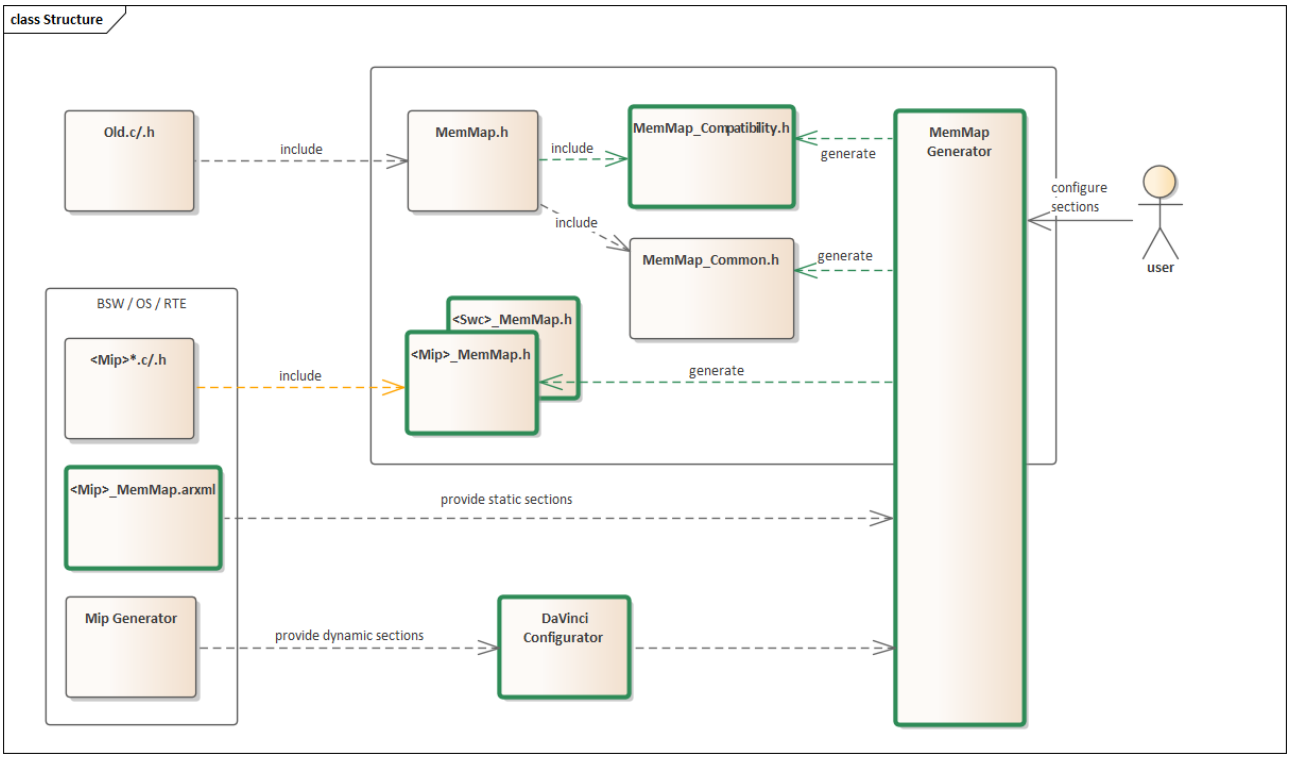


Figure 1 MICROSAR Classic MemMap concept – schematic view

2.1 Common Aspects

Each module shall group and encapsulate objects requiring memory allocation (functions, global constants, global variables) in “section-defining” macros “<MIP>_START_SEC_<specifier>”, and “<MIP>_STOP_SEC_<specifier>”, respectively, before including a Memory Mapping header file.

E.g.:

```
#define MIP_START_SEC_CODE
#include “MemMap.h”
/* function declarations/definitions */
#define MIP_STOP_SEC_CODE
#include “MemMap.h”
```

The “<specifiers>” parts are defined by AUTOSAR, but additional specifiers may be used as well. For this documentation, the exact specification is not important.

A “MemMap” header file has some important properties, compared to regular header files:

- No multiple inclusion Protection. MemMap header files need to be included multiple times from a single Translation Unit.
- Does not contain definitions nor declarations.
- Merely consists of pre-processor switches #ifdef/#if defined(..), #elif.
- `#undef`s a section start or stop macro, if it was handled.
- May contain (compiler specific, section defining) #pragmas, depending on concrete settings.

2.2 “MemMap.h”-based components

Before AUTOSAR 4.1, the Memory Mapping is realized by a single, monolithic include file, “MemMap.h”.

Modules include this MemMap.h file. All section defining macros (see above) used in those modules are handled by the MemMap.h file.

The MemMap.h file (partially) resembles the Memory Mapping configuration; by using #pragmas the compiler is instructed to emit appropriate sections.

In MICROSAR product, the default configuration aggregates and groups modules’ common memory sections (such as CODE, CONST_<alignment>, …) and re-maps them to appropriate section keywords. They are handled, and can be configured, in “MemMap_Common.h”.

An integrator may:

- Configure groups of Memory Sections, via “MemMap_Common.h”
- Configure/map individual memory Sections in “MemMap.h”
- Map Memory sections to custom groups of Memory sections, by defining new ones and using them in “MemMap.h”

However, it is not necessary to directly edit “MemMap_Common.h” anymore. Instead, these common sections can be configured via module “MemMap” in DaVinci Configurator Classic, too.

Within the MemMap.h file a section might be re-mapped to another, module-defined section, creating a dependency between modules’ section definitions. To resolve dependencies from “MemMap.h”-based components on components based on section descriptions, a “MemMap_Compatibilty.h” file may be generated, which should be included into MemMap.h. However, such dependencies are legacy style. They can also be avoided by defining memory “common” memory sections, mapping to them and using DaVinci Configurator Classic to do the specific configuration.

2.3 Components with section descriptions

Since AUTOSAR 4.1, AUTOSAR modified the Memory Mapping concept in two important points:

1. Memory Mapping shall be configurable, using a configuration tool. Same methodology as Module configuration itself shall be used, i.e., modules describe themselves (here: Memory Sections) in a module description, and configuration settings are stored in an ECU configuration.
1. Modules shall include a specific “<Mip>_MemMap.h”, instead of the large “MemMap.h” (dealing with all MemMap sections in a project). This file shall be generated, based on configuration.

(AUTOSAR) Modules, describe their memory sections in their module description. MICROSAR Classic modules supplement their “<Mip>_bswmd.arxml” file with a “<Mip>_MemMap.arxml”. Additionally (also alternatively), memory section descriptions may be created on-demand during configuration, based on a module’s configuration. This feature enables support of multi-partition and multi-core configurations, which highly depends on configuration. DaVinci Configurator Classic stores such “dynamic” description parts in separate files.

Each memory section’s description references an existing SwAddrMethod. MICROSAR Classic provides a standard set of SwAddrMethods used by all modules. Additional Methods may be created automatically by OS Configuration (“OS Applications”) and are referenced by “dynamically” created memory sections. Since those SwAddrMethods depend on configuration, they are never referenced by static sections, defined in (delivered) module descriptions.

2.3.1 Conceptual Overview

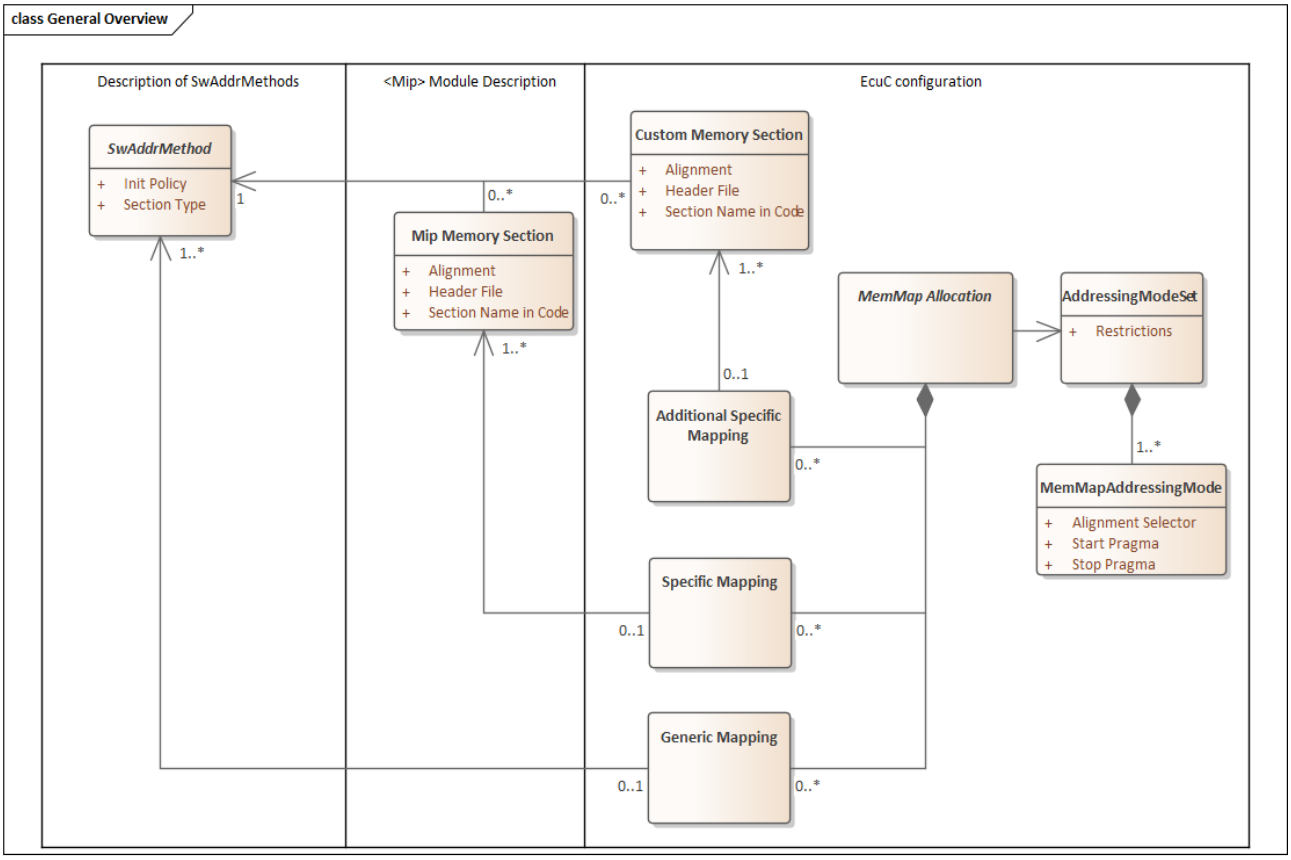


Figure 2 – Conceptual overview of description and configuration elements for Memory Mapping

Figure 2 depicts the key elements involved in Memory Mapping at configuration time. It depicts the overall concept; for brevity reasons it only shows most important elements. The real structure is more elaborated.

2.3.1.1 SwAddrMethods

SwAddrMethods group memory sections into sort of categories, distinguished by their names (more precisely: fully qualified names, aka paths). Each one determines the type (Code, Const, Variables) and the initialization policy (sections containing variables only).

SwAddrMethods are described in separate AUTOSAR Description files. MICSROSAR Classic contains one file describing all default Methods, which are used by MICROSAR Modules.

Additionally, during configuration, modules (e.g. MICROSAR OS) may cause definition of new SwAddrMethods. They are stored into ECUC specific “dynamic” description files, similar to other descriptions which depend on configuration; e.g. SWC Descriptions of Service Components. In this documentation, those SwAddrMethods are called “dynamic”.

Finally, integrators may supply user-defined descriptions of their specific SwAddrMethods.

© 2022 Vector Informatik GmbH

Note
AUTOSAR Descriptions support the notion of packages. Therefore, user-supplied SwAddrMethods may have identical names, if the packages (and hence, the paths to address them) they're located in are distinguishable.

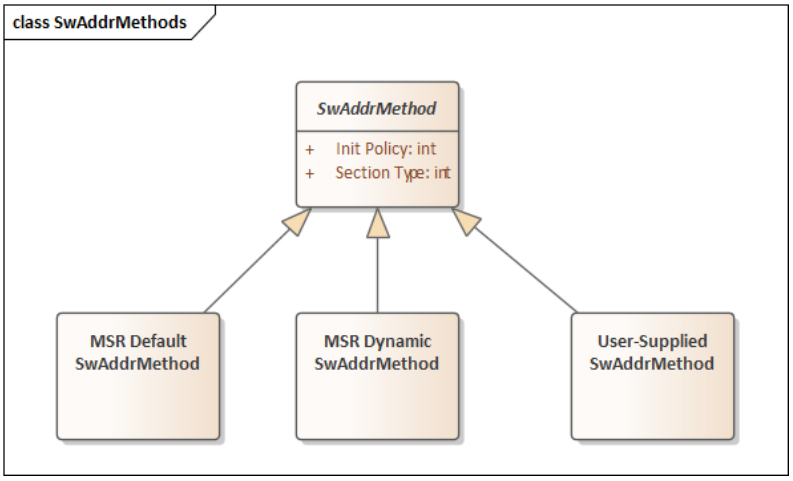


Figure 3 – Types/Sources of SwAddrMethods

Figure 3 summarizes the different sources of SwAddrMethods.

2.3.1.2 <Mip> Memory Sections

AUTOSAR compliant modules shall describe their memory sections in their module descriptions. Each memory section entry:

- Has a name (as every identifiable object in AUTOSAR Descriptions)
- Indirectly identifies the MemMap file, the section #pragma(s) shall be generated to
- Indicates the (macro) names ("Start" and "Stop") used in code; these names need to be generated into a MemMap header file. They are not described explicitly; AUTOSAR specified a recipe to derive the names from given information.
- Data (i.e. const and variable) sections additionally contain alignment information.

Most memory sections are fixed by implementation. However, in some cases they depend on configuration settings, e.g., whether a module's multi core/multi partition support is enabled.

Throughout this documentation, where needed, memory sections contained in a module's supplied description are called "static" memory sections; memory section descriptions resulting from specific configuration are called "dynamic" memory sections.

2.3.1.3 Custom Memory Sections

Custom memory sections[1] allow adding memory sections which are not described in any module's description file. The sections used by "MemMap"-based modules are one example. Another use-case might occur, when an integrator needs custom sections, which are not

[1] This is a MICSROSAR specific extension to AUTOSAR 4.x

assigned to a specific module. Defining them in configuration is more convenient than creating a (“virtual”) module along with a module description.

Custom memory sections contain information very similar to those defined by <Mip> memory sections in a module description.

2.3.1.4 Addressing Mode Sets

An “Addressing Mode Set” defines restrictions regarding which kinds of memory sections may be used along with the mode set. E.g., a mode set may (and typically will) care about code sections only. Another mode set might only accept initialized data sections.

By aggregating one or more “Addressing Modes”, the mode set finally describes which section defining #pragmas shall be used for which concrete section(s).

For data (const, variable) sections, the “Addressing Mode” may contain an alignment selector; this allows separating differently aligned sections from each other, if necessary.

2.3.1.5 MemMap Allocations

An “Allocation” creates the association between individual MemMap sections and/or SwAddrMethods to addressing mode sets.

Each SwAddrMethod or Memory Section shall be referenced by at most one MemMap Allocation.

Unused SwAddrMethods (i.e., not referenced by memory sections) don’t need to be mapped. A MemMap section doesn’t need to be referenced by a “MemMap Allocation”, if its SwAddrMethod is referenced. In that case, it’s mapped via that generic mapping.

On the other hand, if both, a MemMap section and its SwAddrMethod, are mapped, the section’s mapping takes precedence, because it is more specific than SwAddrMethod’s mapping.

The mapping sub-containers of “MemMap” Allocation, depicted in Figure 2, describe the classes of references that can be established. They form the three kinds of elements that may be mapped:

- SwAddrMethods
- <Mip> sections
- User-defined sections

All three kinds of mapping may occur simultaneously in a single MemMap Allocation. This allows a general mapping of SwAddrMethods together with specific sections to the same “Addressing Mode Set”.

2.3.2 Module “MemMap”

Module “MemMap” provides configuration, i.e. mapping, facilities, and is responsible for generation of all MemMap include files:

- <Mip>_MemMap.h
- (optional) MemMap_Compatibility.h
- (optional, recommended) MemMap_Common.h

To enable configuration and generation of MemMap_Common.h, the module allows to describe arbitrary “virtual” memory sections to be generated into arbitrarily named header files. “MemMap_Common” is one of them, and it is part of our recommended configuration. This allows configuring nearly all aspects of MemoryMapping in DaVinci Configurator Classic, independent of the concept different modules implement. Memory mapping for modules, still implementing AUTOSAR 3.x MemMap-concept, can be kept consistent with settings for those implementing AUTOSAR 4.3 concept.

2.4 vLinkGen

vLinkGen is a Vector-specific module that completes the section-defining workflow towards target platform and build toolchain (compiler + linker).

Note
vLinkGen is part of MICROSAR Classic’s “vBaseEnv” package. Usage in conjunction with MemMap is optional; there’s no requirement to configure Memory Mapping via vLinkGen, though it is recommended as it is much more convenient.

It augments the model depicted in Figure 1 as shown below.

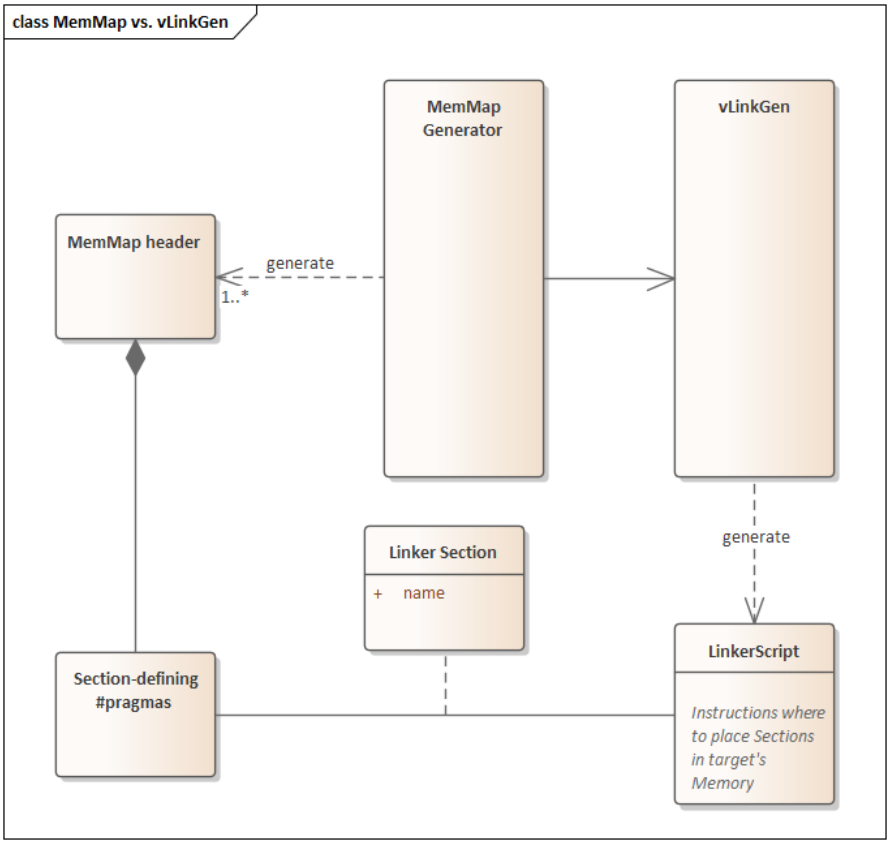


Figure 4 MemMap Module (Generator) and vLinkGen (schematic)

If target platform and build tool are supported, vLinkGen can be used to:

> Define target’s memories/memory areas. Target’s built-in memory areas are provided as a pre-configuration.

- Map sections (to be) emitted by compiler to those memory areas
- Generate tool-specific linker scripts reflecting this mapping
- Associate MemMap sections (Addressing Modes), with vLinkGen sections, resulting in deriving appropriate section-defining #pragmas in MemMap's generated files.

vLinkGen's configuration is portable: It can be imported into other projects for other platforms/toolchains. There it generates appropriate scripts as well.

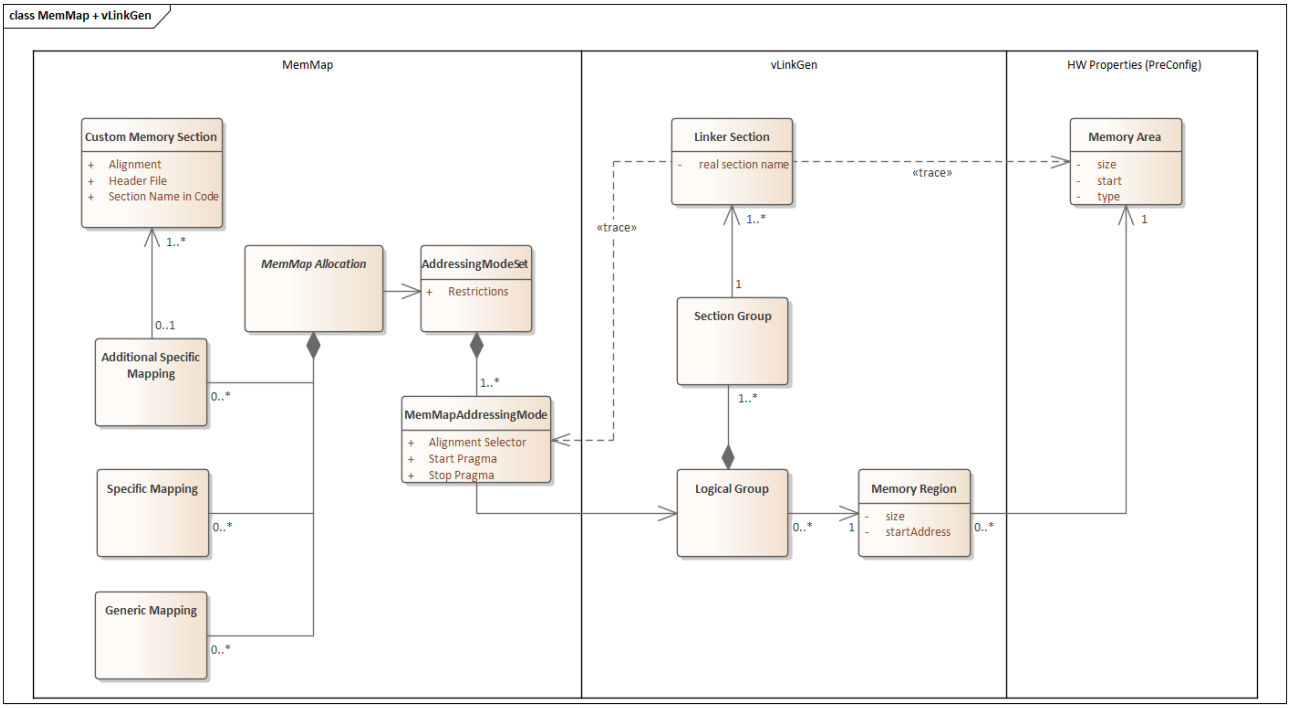


Figure 5 Relationships between Memory Mapping and vLinkGen

Figure 5 shows the relationships between MemMap's and vLinkGen's key configuration elements.

To associate memory sections with a concrete target memory area, the addressing mode just references a Logical Group defined in vLinkGen. So, MemMap generator gets knowledge of the #pragmas to be generated for associated MemMap sections into MemMap header files, according to vLinkGen's configuration of Section Groups and Linker Sections.

vLinkGen defines output section names to be generated by MemMap along with the compiler specific #pragmas.

Note
Explicit #pragmas, configured in "Addressing Mode" take precedence over those derived from vLinkGen. This way, it is possible to individually override settings.

3 Configuration

Configuration of memory mapping is done using DaVinci Configurator Classic.

3.1 Default configuration of MemMap

The module MemMap is shipped with a recommended configuration, which contains default mappings for all SwAddr Methods defined for MICROSAR Classic.

Those mappings default to empty #pragmas, i.e. compiler defaults will be used during build:

- All code sections (usually) map to “.text” sections
- All const sections (usually) map to “.const”. Compilers also might choose to emit constants along with code into “.text”
- Uninitialized data and cleared (zeroed) data are emitted to “.bss” sections
- Initialized data map to “.data”; embedded compilers usually additionally emit an “.init” section, containing variables’ initial values, which shall be copied to “.data” by startup code.

3.2 MemMap.h

In a MemMap.h file, sections should be handled in a way that they can be configured in DaVinci Configurator Classic as well. The “MemMap.h” template, as delivered, “forwards” all memory sections to common sections. These are defined/handled in “MemMap_Common.h”. This principle should be kept for more specific sections mappings: Instead of adding #pragmas directly to “MemMap.h”, appropriate secondary section (or group) names should be chosen.

Then, these names should be used to configure MemMap_Common.h as described in the following paragraph.

3.3 MemMap_Common.h

The default (recommended) configuration of MemMap module also provides entries which resemble the default contents of “MemMap_Common.h”.

This configuration is a set of “custom memory sections” and is located in container “MemMapModules”.

Additional sections to be generated to “MemMap_Common.h” can be added to that container. Usually this is necessary when partitioning is required.

© 2022 Vector Informatik GmbH



FAQ
A recommended configuration will not be loaded if a module configuration already exists. To force loading MemMap's recommended configuration, disable module MemMap in DaVinci Configurator Classic's project settings. Committing (saving) this change removes existing settings – they will be lost. Then, re-enable it.

3.4 Input to Configuration

The input to the configuration is part of module description. It may be either static (given by the description files) or dynamic, depending on module-specific configuration settings.

In the "static" case, memory sections are fixed; they don't depend on a module's configuration. "Dynamic" memory sections become necessary when modules explicitly support multi-core and/or multi-partition use cases. There is a virtually unlimited number of scenarios for such use cases. MICROSAR Classic modules are very flexible in supporting them. Consequently, memory sections cannot be statically described; the ability of dynamically adding and removing memory section descriptions and SwAddrMethods during configuration is a cornerstone in MICROSAR Classic's Memory Mapping concept.



Expert Knowledge
Memory Sections' descriptions are contained in modules' internal behaviors. Dynamically created descriptions are synchronized to <project-path>\Config\InternalBehavior\<Mip>_ib_bswmd.arxml

3.5 Configuration Workflow

During initial project setup no special mapping should be required, defaults should be sufficient. However, as project progresses, and its configuration gets refined, specific mappings might become necessary, quite early. Especially, with multi-core targets, when software components need to be mapped to specific cores, additional dynamic SwAddrMethods or sections might be created. Unmapped memory sections cause validation errors in MemMap module; they need to be mapped. Therefore, it is advisable to eagerly create an initial set of "mapping targets", i.e. Addressing Mode Sets.

3.5.1 Target Hardware

The target HW implies constraints and requirements on Memory mapping.

1. CPU Cores. Multicore systems usually require specific per-core mappings for:
    a. Exclusive Memory areas: data/variables which are accessed exclusively by one CPU core.

   b. Shared memory dedicated to sharing data between a defined set of CPUs (which CPUs may access such an area);

2. Physical memory layout: Are there any special memories, certain elements need to be or should be located in? Examples are special local (tightly coupled) and shared (between different CPU cores) RAM areas, fragmentation of memory areas (not contiguously addressable).
   a. Fast, tightly coupled RAM is often a very limited resource. Data to be stored here might exceed available RAM. There might be the necessity to use slower global RAM portions for CPU-exclusive data access. They should be separated from shared data areas.
   b. CPU Caches might affect the mapping to memory areas: Some areas may/should be cacheable, while others should be excluded from caching. E.g. a CPU-exclusive memory area should be cacheable, while shared memory should not. Other areas might just in a way where performance does not matter, and the cache shall not be polluted with data from those sections. I.e., more important data requiring fast accesses shall not be replaced in cache by less important data.
   c. HW peripherals directly accessing dedicated RAM areas (DMA capabilities). A prominent example are Ethernet controllers, writing received data to and reading data to be transmitted from dedicated buffer areas.

These considerations should result in a basic set of “Addressing Mode Sets” and subordinated addressing modes. If vLinkGen is used, configuration could also start there, by defining respective linker sections, section groups and logical groups.

Refer to [4] for more information about Multi Core support in MICROSAR Classic.

3.5.2 Partitioning, ASIL Levels

Software parts with different safety Levels (QM, ASIL A … D) must run isolated from each other, in order to guarantee freedom from interference. (Mutable) data of higher ASIL levels must be protected from write accesses by lower levels.

This separation is commonly called “Partitioning”.

For more information on partitioning see [5].

3.5.2.1 Cross-Partition Data Exchange

There are certain cases MICROSAR Classic Modules need to access same data from different partitions. To ensure freedom from interference, such data need to be write-protected for lower ASIL partitions, while highest-ASIL partition may write them. Special requirements on protection of memory areas are documented in Safety Manual. It denotes the memory sections that shall be protected.

Such sections are identified by their names.

> Either, the name used in Module’s implementation, and defined in MemMap.h.

> Or, the unique path in Module’s description, which can be directly referenced in MemMap configuration.

These memory sections need to be explicitly mapped appropriately. Concrete mapping depends on Software Architecture, Safety Concept, and other constraints.

The specific mapping to appropriate area must be reflected by:

- A “MemMap Addressing Mode”, contained in appropriate “Addressing Mode Set”
- (Optional) a suitable “MemMap Allocation”. (“MemMap Allocations” are a facility to group mappings; they do not affect the mapping itself.)
- “Specific Mapping” or “Additional Specific Mapping” associating the memory section with the appropriate “MemMap Addressing Mode”

As with memory mapping in general, it must be ensured that specified sections will be located in right memory areas. This is achieved by defining right #pragmas for the “Addressing Mode Sets” and Linker configuration (e.g. using vLinkGen).

Finally, memory protection of address areas, i.e. MPU configuration, is done in OS.

3.6 Post-Build Configurations

Post-Build Loadable use cases requires a particular memory layout regarding configuration data. This must be considered when configuring Memory Mapping. For more information on that topic, refer to [1].

3.7 Creating the Mapping

Once the (initial) mapping targets, i.e. Allocations and Mapping Mode Sets, have been defined, individual sections or groups of them (SwAddrMethods) can be assigned to them. This is done by either:

- Creating new “MemMap” Allocations and new Mappings.
- Creating new Mapping within existing “MemMap Allocations”
- Adding new references to existing Mappings

The different ways allow very flexible and specific grouping. However, there must be at least one “MemMap Allocation” per “Addressing Mode Set”, as only this allows for correct assignment of sections to their intended areas.

Initially, it is usually sufficient to map only SwAddrMethods. This way, all memory sections get mapped. Then, individual memory sections can be mapped to their required areas, as this overrules generic, SwAddrMethod-based mapping.

4 Glossary and Abbreviations

4.1 Glossary

Term	Description

4.2 Abbreviations

Abbreviation	Description
MPU	Memory Protection Unit
Mip	Module Implementation Prefix

5 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2022 Vector Informatik GmbH